XE

XELOR · BUILD AND DELIVERY BLUEPRINT

Architecture, Delivery Pipeline & Implementation Playbook

The executable “how”: technology choices, repository changes, CI/CD, infrastructure, data, security, observability, testing, recovery, ownership and phased delivery.

Prepared from the live repository and deployment package

4 August 2026 · v1.0

1. The decision in one page

Build XELOR as a boundary-enforced modular monolith and operate it as three independently scalable containers—Web, API and Worker—until real scale or team ownership proves that a module must be extracted. The next investment should make releases repeatable, data recoverable and behavior observable; it should not add microservices or Kubernetes.

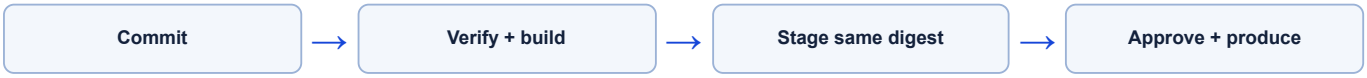
Recommended production shape: AWS Mumbai primary; CloudFront/WAF and an Application Load Balancer; ECS Fargate Web/API/Worker; ECR; RDS PostgreSQL Multi-AZ; managed Valkey; S3/KMS; Secrets Manager; Keycloak behind the OIDC boundary; OpenTelemetry into CloudWatch/Grafana; GitHub Actions using short-lived AWS identity.

Keep

- Node.js 22, TypeScript, pnpm workspaces.
- Next.js 15 / React 19 Web.
- NestJS 11 modular API and shared platform rules.
- PostgreSQL 17, SQL migrations and FORCE RLS.
- Valkey/BullMQ and transactional outbox.
- Keycloak OIDC + PKCE for non-demo environments.
- Deterministic AI stub until each AI use case passes evaluation.

Add now

- GitHub pull-request and deployment workflows.
- Immutable, scanned images promoted by digest.
- OpenTofu infrastructure modules and remote state.
- One-shot migration task and expand/contract rules.
- Runtime readiness, graceful shutdown and deployment alarms.
- Traceable structured telemetry and SLO alerting.
- Restore, rollback and incident runbooks exercised on a schedule.



3	1	0	1
runtime application roles	transactional system of record	static AWS access keys	promoted digest per image

Three prerequisites before the first production deployment: replace the public build-time API origin with runtime/private routing; replace demo-file readiness with dependency/schema readiness; and add the missing checked-in GitHub workflows. These are implementation gaps, not reasons to rewrite the product.

2. What was verified in the repository

This playbook is based on code and configuration present in `MVP_PROTOTYPE_1`, not on a hypothetical greenfield stack.

Area	Repository evidence	Architectural meaning
Workspace	`apps/web`, `apps/api`, `packages/platform`, `packages/db`; pnpm 9; Node 22; TypeScript 5.7.	One typed codebase with explicit runtime and domain layers.
Web	Next.js 15, React 19, Tailwind 4, Radix UI, Playwright.	Server-capable UI that should remain a client of the API, never the database.
API	NestJS 11; 28 controller surfaces; Zod validation; permission and tenant guards.	A modular monolith already exposes a credible enterprise API boundary.
Data	PostgreSQL 17, Drizzle, 64 forward SQL migrations, pgvector/pg_trgm/btree_gist and forced tenant RLS.	Transactional data integrity and tenant isolation belong in PostgreSQL as well as application code.
Async	Transactional outbox, BullMQ, Valkey and idempotent worker patterns.	Effects can be retried without pretending the queue is the source of truth.
Identity	Keycloak 26, authorization code + PKCE, JWKS verification and in-application permission registry.	The OIDC boundary is portable; the demo bypass must remain isolated.
AI/agents	7 agents, 16 capabilities, 5 graphs, durable checkpoints, approval gates, evidence and deterministic provider stub.	AI can be introduced behind governance and evaluation without giving a model direct ERP writes.
Quality gates	Lint, typecheck, module manifest check, 727 service/unit tests, DB RLS/permission checks, 99-item live acceptance matrix and browser scenarios.	Existing local checks can become enforceable release policy.
Deployment	Local Compose plus five-service Railway demo packaging and health/start scripts.	Demo packaging exists; production delivery and IaC do not yet.

Gaps this document closes

Observed gap	Required correction	Proof of completion
No `.github/workflows` release pipeline was found.	Add CI, security, staging, production and recovery workflows described here.	A protected `main` cannot merge or deploy without named checks.
`NEXT_PUBLIC_API_ORIGIN` is read by `next.config.ts`.	Route `/api/v1` at the ALB or use a server-only runtime origin; remove the public build dependency.	The identical Web image digest runs in staging and production.
Readiness depends on a demo seed file when configured.	Add distinct live, ready and startup behavior; readiness checks DB, schema compatibility and required broker connectivity.	ECS never sends traffic before dependencies are usable; seeding is not a production health condition.
API startup applies migrations in the Railway demo.	Run production migrations once as a separately authorized ECS task.	Scaling API tasks cannot race migrations or require owner credentials.

3. Environment model and promotion contract

Environment	Purpose	Identity/data	Deployment	Promotion rule
Local normal	Daily development and realistic identity testing.	Compose PostgreSQL, Valkey, Keycloak, Gotenberg; seeded local data.	Web/API/Worker run from pnpm or parity containers.	Developer branch only; no external data.
Local demo parity	Reproduce the public investor topology.	Disposable data; fixed public-demo persona.	`infra/railway/docker-compose.demo.yml`.	Required before changing Railway boot/deploy files.
Public demo	Anyone can view a controlled investor story.	Disposable seeded data only; explicit demo bypass; no customer integrations.	Railway Web/API/Worker/PostgreSQL/Valkey.	Separate flags and database; never promoted into staging.
Staging	Production-like acceptance and migration rehearsal.	Real OIDC; synthetic/de-identified data; same schema and service topology.	Dedicated AWS account/VPC and production class where feasible.	Receives immutable digests built from `main`.
Production	Pilot/customer operations.	Real tenant data; MFA; demo flags prohibited.	Separate AWS account, Mumbai primary.	Only a digest that passed staging and human approval.
Recovery	Restore and regional failover exercises.	Encrypted copies and restricted operators.	Hyderabad or an approved alternative; warm infrastructure as RTO requires.	Created/tested from IaC, not configured manually.

> **Core promotion rule:** build Web and API images once for a commit, record their digests, and promote those exact digests. Configuration and secrets change by environment; application bytes do not.

Configuration classes

Build-time

Compiler flags and static asset decisions only. No environment URL, secret, tenant or feature authority.

Runtime

Service endpoints, pool sizes, log level, feature rollout and resource tuning. Supplied by task definition.

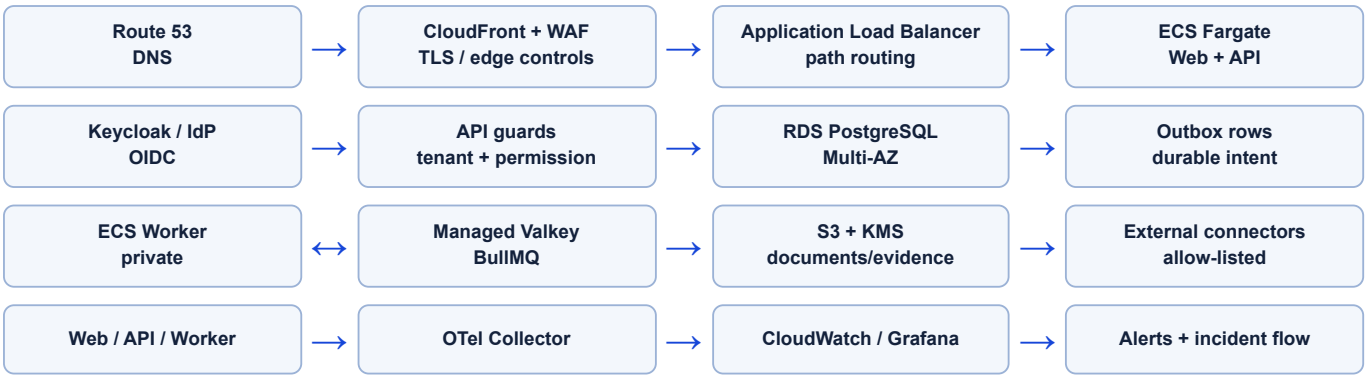
Secret

Database passwords, OIDC client credentials, connector keys and signing material. Referenced from Secrets Manager.

Hard environment invariants

- `API_PUBLIC_DEMO` and `NEXT_PUBLIC_PUBLIC_DEMO` must be absent or false in staging and production. Add a startup assertion that aborts when a production environment sees either true.
- Production application tasks receive the restricted `app_user` database URL. Only the migration task role can resolve the owner URL.
- No customer production data is copied to public demo or developer laptops.
- Every production environment change is represented by reviewed application or infrastructure code, with an audit trail.

4. Target production architecture



Layer	Implementation	Why this choice	Initial topology
Edge	Route 53, ACM, CloudFront, WAF, ALB.	Managed TLS, rate/filter rules, caching for static assets and health-aware routing.	One public distribution; only ALB ingress from the edge where practical.
Compute	ECS Fargate services for Web/API/Worker; ECR images.	Independent scaling without a Kubernetes control plane.	At least two Web and API tasks across AZs; Worker count driven by queue.
Data	RDS PostgreSQL Multi-AZ with encryption, PITR and alarms.	Managed failover and recovery for the transactional system of record.	Private subnets; no public endpoint; measured connection pool.
Queue/cache	Managed Valkey compatible with BullMQ.	Preserves existing async semantics while removing single-container state risk.	Private; TLS/auth; queue persistence tuned to recovery needs.
Documents	S3 buckets, KMS CMKs, object versioning and lifecycle policies.	Container disks are ephemeral and not an evidence store.	Separate raw/quarantine/approved/export prefixes or buckets.
Identity	Keycloak initially, isolated behind the standard OIDC contract.	Avoids application auth rewrite; preserves enterprise federation path.	At least two tasks and a dedicated database for pilot; managed IdP later if operations justify it.
Telemetry	OpenTelemetry Node instrumentation and Collector; CloudWatch/Grafana; browser error tracking.	Vendor-neutral traces/metrics and actionable operational evidence.	Node traces/metrics first; avoid depending on experimental browser OTel.

Not selected now: EKS/Kubernetes, service mesh, Kafka, separate graph database and a warehouse are deferred until measured scaling, fan-out, query or ownership needs justify their operational cost.

5. Network, request and trust boundaries

Public edge	Browser → CloudFront/WAF → ALB. Only HTTPS 443. WAF rate, reputation and managed rules; request-size limits; TLS certificate in ACM.
Application	ALB sends `/api/v1/*` to API and all other routes to Web. Web and API live in private subnets. API validates signed token, tenant and permission on every protected route.
Data	Security groups allow PostgreSQL only from API/Worker/migration tasks and Valkey only from API/Worker. No database or broker public address.
Egress	Connector/AI calls originate through controlled NAT or VPC endpoints, use allow-listed destinations and carry no broader tenant data than the approved contract.
Operations	Engineers use SSO and audited AWS roles. No SSH into tasks. ECS Exec, if enabled for incidents, is time-bound, logged and disabled by default.

Correct same-origin routing

The browser should never know an internal API hostname. This eliminates CORS complexity, keeps one public origin and lets the same Web image run everywhere. Keep a local Next.js rewrite for developer convenience, but use a server-only `API_ORIGIN` or ALB path routing—not `NEXT_PUBLIC_API_ORIGIN`—outside local/demo builds.

Request contract

1. Edge assigns or preserves a safe request identifier; API starts a trace and returns its `traceId` in the error envelope.
2. API verifies issuer, audience, signature, expiry and allowed algorithms from OIDC JWKS; tenant comes from the verified identity, never a browser-selected header in normal mode.
3. Route permission and separation-of-duties checks run before business logic.
4. Mutation payload is schema-validated and carries a stable `Idempotency-Key`.
5. Domain service opens a tenant transaction, executes `SET LOCAL app.current_tenant`, writes business/audit/outbox rows and commits atomically.
6. Response exposes stable identifiers and evidence, not raw secrets or internal exception messages.

6. Repository structure to add

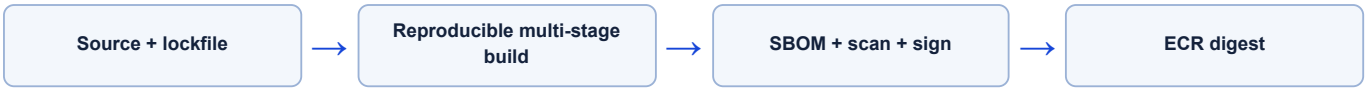
```
.github/
  CODEOWNERS
  workflows/
    ci.yml                    # PR correctness gate
    security.yml              # dependencies, SBOM, image/IaC scanning
    deploy-staging.yml        # build once, migrate, deploy, verify
    deploy-production.yml     # approved promotion of same digests
    restore-drill.yml         # scheduled non-production recovery proof

infra/
  docker/
    Dockerfile.web            # production multi-stage image
    Dockerfile.runtime        # API + Worker shared image
  opentofu/
    bootstrap/                # remote state bucket/lock/KMS only
    modules/
      network/ edge/ ecs-service/ rds/ valkey/ identity/ telemetry/
    envs/
      staging/ production/ recovery/
  railway/                    # keep disposable public-demo packaging

scripts/
  release/
    assert-environment.mjs    # prohibit demo flags and invalid config
    migration-plan.mjs        # report pending/unsafe migrations
    smoke.mjs                 # health + critical contract probes
    verify-digests.mjs        # staging/production digest equality

docs/runbooks/
  deployment.md  rollback.md  database-restore.md
  queue-recovery.md  identity-outage.md  incident-response.md

apps/api/src/observability/
  bootstrap.ts  logging.ts  metrics.ts  redaction.ts

apps/api/src/health/
  health.controller.ts  readiness.service.ts  schema-compatibility.ts
```

Ownership rules

Path	Required reviewer	Reason
`packages/db/migrations/**`, RLS and permission registry	Backend/data owner plus security-aware reviewer.	Schema, tenant isolation and deploy compatibility are system-wide contracts.
`packages/platform/**`	Owning domain engineer.	Business invariants and module boundaries live here.
`apps/api/**`	Backend owner; security review for auth/effects.	Public behavior, authorization and transactions.
`apps/web/**`	Frontend/full-stack owner; design/accessibility check.	User contract and production bundle.
`infra/**`, `.github/**`	Platform owner; second approver for production.	Changes runtime authority, network or release controls.

Branch model: short-lived branches, pull request into protected `main`, required checks and one reviewed deployment path. Avoid long-lived environment branches; environments are configuration and promotion targets, not divergent code lines.

7. Container and artifact pipeline



Images

Image	Contents	Runtime command	Rules
`xelor-web:<sha>`	Next.js production output and static assets only.	Start Web on port 3001.	Non-root, read-only root filesystem where supported, no source maps with secrets, no environment-specific public API URL.
`xelor-runtime:<sha>`	Compiled API, Worker, platform and DB runtime/migration code.	API service, Worker service or one-shot migration command.	Same image for API/Worker/migration; different task roles and commands.

Build requirements

- Pin base images by digest after an update process; use Node 22 LTS-compatible runtime and Corepack/pnpm 9.12 with frozen lockfile.
- Builder receives dev dependencies; runtime stage receives only production output and the minimum migration assets required by its command.
- Run as a numeric non-root user; set a restrictive filesystem and Linux capability policy; place writable temp files only in declared paths.
- Attach OCI labels for commit, repository, build time and version. Tag by commit SHA for readability, deploy by digest for immutability.
- Generate CycloneDX or SPDX SBOM, scan OS/npm/IaC issues, sign the image with keyless workload identity where supported, and retain provenance.

Artifact acceptance

✓	Cold start succeeds with a non-root user and read-only filesystem policy.
✓	Image contains no `.env`, Git metadata, test fixtures, customer exports or owner database credential.
✓	Web and runtime digests are recorded in a signed release manifest.
✓	The same digests are used for staging and production; only task configuration differs.
✓	Critical vulnerabilities fail release unless a time-bounded, approved exception exists.

8. Pull-request CI pipeline

PR opened → Static + unit gates → Real service integration → Build + report

Job	Exact work	Failure means
Changes	Classify Web/API/platform/DB/infra/docs changes and calculate jobs, without skipping global dependency files.	Pipeline cannot safely determine scope; run the full suite.
Install	Checkout, Node 22, Corepack, pnpm 9.12, `pnpm install --frozen-lockfile`, dependency cache keyed by lockfile.	Build is not reproducible.
Static	`pnpm lint`, `pnpm typecheck`, module-manifest/boundary validation, formatting check if adopted.	Code or architecture contract is invalid.
Unit	`pnpm test`; upload machine-readable test report and coverage trend.	Business/service contract regressed.
Database	Start PostgreSQL/Valkey/Keycloak parity services; run migrations from empty DB; RLS check; permission reconciliation; migration idempotence/ordering check.	Schema, isolation or permission contract is unsafe.
Integration	Start API/Worker/Web with test identity; run the 99-item live matrix and API contract probes; confirm outbox consumption.	The assembled system does not behave like its units.
Browser	Playwright critical journeys on Chromium on every PR; broader browser/responsive matrix nightly or before release.	A user cannot complete a protected critical journey.
Build	Build production Web/runtime images, boot them, run health probes and scan filesystem.	Passing source cannot produce a runnable artifact.
Security	Dependency, secret and IaC/container scanning; CodeQL or equivalent scheduled/on PR for sensitive changes.	Known release-blocking exposure or leaked secret.

Required branch checks

```
ci / static
ci / unit
ci / database-contracts
ci / integration
ci / browser-critical
ci / image-build
security / policy
```

> **No “temporary” bypass:** an emergency merge exception must be restricted to named administrators, create an audit event and require a follow-up review. It cannot silently turn a failing check green.

9. GitHub workflow design

`ci.yml`

```
on: pull_request, push(main)
permissions: contents: read
jobs:
  static -> unit -> database-contracts -> integration -> browser-critical
  image-build (parallel after static; boots final images)
  ci-success (depends on every required job)
```

`deploy-staging.yml`

```
on: successful CI for main, or manual dispatch
permissions: contents: read, id-token: write
environment: staging
steps:
  authenticate to AWS with GitHub OIDC
  build/sign/push Web + runtime images once
  write release manifest containing immutable digests
  run pre-deploy compatibility checks
  run one-shot migration task
  deploy backward-compatible Worker -> API -> Web using those digests
  wait for ECS service stability and circuit-breaker result
  run smoke + 97 acceptance checks
  store manifest, results and deployment metadata
```

`deploy-production.yml`

```
on: manual promotion of a passed staging release manifest
permissions: contents: read, id-token: write
environment: production (required reviewers)
steps:
  verify commit, signature, scan status and staging evidence
  verify ECR digests exactly match staged digests
  snapshot/PITR and migration preconditions
  run production migration task
  roll backward-compatible Worker -> API -> Web with circuit breaker + alarms
  run non-destructive smoke tests
  watch deployment SLOs for the defined observation window
  record success or execute the rollback decision tree
```

OIDC trust conditions

- GitHub receives `id-token: write`, not a stored AWS key.
- AWS role trust restricts audience and the token `sub` to this repository and named GitHub Environment.
- Staging and production use different roles/accounts; production has narrower permissions and approval protection.
- Workflow pins third-party actions to reviewed commit SHAs and uses least-privilege permissions at workflow/job level.

10. Release sequence and deployment safety

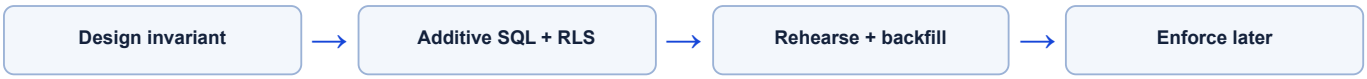
1 · Qualify	All PR checks pass; review complete; merge commit is immutable input.
2 · Build	Build Web/runtime once; scan, sign, push; create release manifest with digests and migration set.
3 · Rehearse	Deploy same digests to staging; migrate from a production-like prior schema; run integration, browser and acceptance probes.
4 · Approve	Human reviewer checks change, risk, database plan, metrics, rollback safety and staging proof.
5 · Migrate	One-shot ECS task uses the owner secret, obtains an advisory release lock, applies migrations once, emits evidence and exits.
6 · Roll	Deploy API, Worker and Web in compatibility order; ECS rolling deployment has circuit breaker and CloudWatch alarm rollback.
7 · Verify	Readiness, non-destructive critical flows, error/latency/queue/DB dashboards, audit continuity and tenant probes.
8 · Observe	Hold the release open for a defined window; record outcome and close only when SLO indicators remain acceptable.

Service order

Change type	Order	Why
Additive schema + compatible application	Migration → backward-compatible Worker → API → Web.	Consumers can handle old/new events before the API produces them; new code sees required structures while old code still functions during rolling overlap.
New consumer of existing outbox event	Worker capable of ignoring/handling both versions → producer/API.	Prevents poison events during mixed versions.
Breaking API response	First make API dual-compatible → deploy Web → later remove old representation.	Users may hold old browser bundles during rollout.
Destructive schema cleanup	Prove all readers stopped → remove references → observe → later migration drops data.	Rollback remains possible during the compatibility window.

Never: deploy application instances that each run owner migrations at startup; rebuild per environment; mutate a running server; or perform a destructive schema change in the same release that removes the last compatible reader.

11. Database change pipeline



Expand / migrate / contract

Release	Database	Application	Rollback status
A · Expand	Add table/column/index/policy; nullable or compatible default; avoid long table rewrite.	Old code unchanged or capable of ignoring the addition.	Old image still safe.
B · Migrate	Backfill in bounded, resumable batches with progress and load limits.	Dual-write or read-new-with-fallback; compare results.	Can return to old read path.
C · Enforce	Add not-null/constraint only after a validating precheck; validate large constraints separately.	New field becomes authoritative.	Requires documented forward fix.
D · Contract	Drop old object in a later release after telemetry proves zero use and retention approval exists.	Old code already impossible to deploy.	Backup/PITR is disaster recovery, not normal rollback.

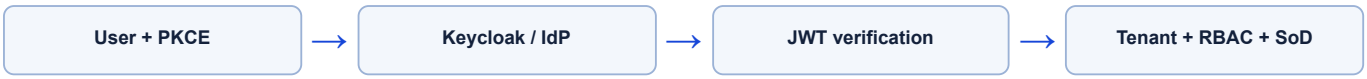
Migration task contract

- Uses `DATABASE_OWNER_URL` only in the one-shot migration task. API/Worker never receive it.
- Takes a PostgreSQL advisory lock keyed to the application to prevent two releases migrating concurrently.
- Reads the repository migration ledger, applies each forward migration once and records checksum, commit, actor, start/end and outcome.
- Sets lock/statement timeouts deliberately. A timeout fails the release; it does not retry blindly.
- Runs RLS, permission and schema-compatibility checks after migration and before traffic reaches new tasks.

Every tenant table must prove

✓	`tenant_id` is non-null and linked to the tenant master where appropriate.
✓	RLS is enabled and forced, and policies use the canonical tenant session setting.
✓	Indexes begin with tenant scope for common tenant-local access patterns.
✓	Uniqueness includes tenant unless the identifier is intentionally global.
✓	Cross-tenant support behavior is an explicit audited capability, never a missing filter.

12. Identity, authorization and secret pipeline



Identity implementation

1. Keep the application OIDC-standard: issuer, audience, JWKS and claims mapping are configuration, not Keycloak-specific business logic.
2. Run the browser authorization-code flow with PKCE. Do not put a client secret in Web code.
3. API pins allowed issuer/audience/algorithms, caches JWKS safely and fails closed if verification cannot be established.
4. Map immutable subject and verified organization/tenant membership. Display names and emails are not authorization identifiers.
5. Resolve route permission in the API and enforce tenant again through database RLS. UI permission hiding improves usability but never grants authority.
6. For pilot, deploy Keycloak HA with at least two application tasks and a dedicated managed database; configure MFA, brute-force controls, backup and realm export procedure.

Secrets lifecycle

Secret	Consumer	Storage/rotation	Exposure rule
Application DB credential	API/Worker.	Secrets Manager; rotate with dual-user or managed pattern after testing pool behavior.	Restricted role; never owner.
DB owner credential	Migration task only.	Separate secret and IAM access policy.	Unavailable to long-running services.
OIDC client credential	Confidential server-side client only.	Secrets Manager; scheduled rotation.	Never a `NEXT_PUBLIC_*` value.
Connector/API key	Named connector worker.	One secret per provider/environment; versioned rotation.	Not shared with general API if avoidable.
KMS/signing key	AWS service or narrowly scoped signer.	KMS/HSM-backed; no export.	Referenced by ARN; least-privilege encrypt/decrypt/sign.

Production guard: application startup must refuse `NODE_ENV=production` when demo bypass is true, issuer is HTTP, default passwords are detected, owner/app database URLs are equal, or mandatory telemetry/redaction configuration is missing.

13. Runtime configuration matrix

Variable / class	Web	API	Worker	Migration	Rule
`NODE_ENV`, `APP_ENV`	Yes	Yes	Yes	Yes	`APP_ENV` explicitly distinguishes demo/staging/production; do not infer authority only from Node mode.
Internal API origin	Runtime server-only/local rewrite	—	Optional health URL	—	Browser uses same-origin. No public internal hostname.
`DATABASE_URL`	Never	app_user	app_user	Optional verification	Web has no database code or credential.
`DATABASE_OWNER_URL`	Never	Never	Never	Required	Only one-shot migration role can retrieve.
`VALKEY_URL`	Never	Yes	Yes	No	TLS/auth in non-local environments.
OIDC issuer/audience	Public issuer/client ID as needed	Required	Only if independently authenticating	No	URLs/IDs are config; secrets are not public.
Demo flags	Demo only	Demo only	Inherited behavior only	No	Startup refusal outside disposable demo.
OTel endpoint/service	Error SDK/runtime config	Yes	Yes	Yes	No sensitive attributes; environment/resource tags required.
Pool/concurrency	—	Measured DB pool	Worker/DB concurrency	1	Total possible connections remain below RDS safety budget.

Validation at boot

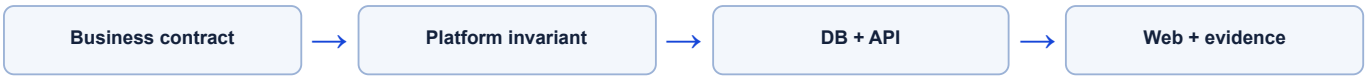
Define one Zod configuration schema per runtime role. Parse once before the server/worker starts; emit only variable names and validation reasons, never values. Validate URL schemes, positive ranges, mutually exclusive modes and production invariants.

Connection budget example

```
safe_app_connections = floor(rds_max_connections * 0.70)
reserved = migration + administration + monitoring + failover headroom
api_pool_max * max_api_tasks
  + worker_pool_max * max_worker_tasks
  + reserved
  <= safe_app_connections
```

Do not copy a pool number from the demo. Load-test query concurrency and use RDS metrics before introducing RDS Proxy or PgBouncer.

14. How to implement one XELOR module safely



Step	Deliverable	Verification before moving on
1 · Story	Actor, trigger, preconditions, authoritative owner, transaction boundary, refusal conditions, evidence and measurable outcome.	Product/domain owner can identify what is committed and what is only proposed.
2 · Boundary	Module manifest, permissions, inbound ports, emitted events and prohibited imports.	Boundary lint fails an intentional forbidden import.
3 · Rule	Pure platform service/value objects with deterministic business rules and typed errors.	Unit/property tests cover happy, boundary, invalid and replay cases.
4 · Schema	Forward migration, keys, constraints, indexes, RLS/force/policies, audit/outbox relationship.	Empty migration, upgrade migration, RLS and concurrent conflict tests pass.
5 · Transaction	One service owns the commit; calls other modules through defined ports; business + audit + outbox are atomic.	Injected failure proves no partial write.
6 · API	Controller, Zod input, permission, idempotency, stable error envelope and OpenAPI/contract representation.	Unauthorized, wrong-tenant, duplicate, invalid and conflict tests pass.
7 · Async	Versioned event and idempotent consumer only for effects that may occur after commit.	Duplicate, retry, poison and replay tests pass.
8 · Web	Navigation manifest, typed API mapping, all UI states, accessibility, evidence/provenance and safe mutation behavior.	Keyboard, loading, empty, denied, error, success and double-click scenarios pass.
9 · Operate	Metrics, trace fields, runbook, alert threshold and support evidence.	A developer can locate one failed request and explain recovery from telemetry.
10 · Accept	End-to-end story and negative-path acceptance probe.	Named owner signs the outcome, not just the screen.

Definition of done: the work is not done when the UI renders. It is done when authority, transactionality, tenancy, replay behavior, failure evidence, user recovery and operating signals have all been proved.

15. Backend developer implementation guide

Write path

1. Controller authenticates, checks route permission, parses Zod input and captures a stable idempotency key.
2. Application service begins the database transaction and establishes tenant context with `SET LOCAL`.
3. Load and lock only the rows required for the invariant; order locks consistently to avoid deadlocks.
4. Evaluate rule in `packages/platform`; turn expected refusal into a stable domain/API code.
5. Persist business state, audit evidence and any outbox event in the same transaction.
6. Commit; return stable resource/version IDs. Post-commit effects are picked up by idempotent workers.

Review checklist

✓	No controller contains business decisions or raw cross-module table choreography.
✓	No tenant query runs outside the tenant transaction helper; RLS is a second control, not a replacement for correct code.
✓	Expected conflicts map to 409/422 with stable codes; unknown faults map to a safe 500 and retain a trace ID.
✓	Mutation replay returns the original semantic result or a defined in-progress state.
✓	External network calls do not occur inside a ledger-critical database transaction unless the pattern explicitly reconciles ambiguity.
✓	Every list endpoint has bounded pagination, deterministic ordering and indexed tenant filters.
✓	No log contains token, password, full document body, bank data or unrestricted request payload.

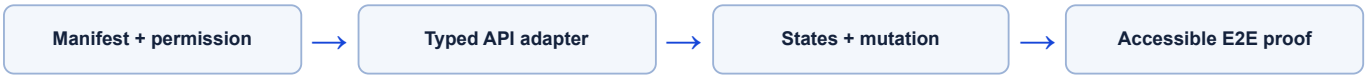
Error envelope

```
{
  "error": {
    "code": "INVENTORY_INSUFFICIENT",
    "message": "Available stock cannot satisfy this issue.",
    "details": { "itemId": "...", "available": "8.000" },
    "traceId": "01J..."
  }
}
```

Expose only safe, actionable details. Preserve the full internal cause in protected telemetry keyed by `traceId`.

16. Full-stack and frontend implementation guide

Screen delivery sequence

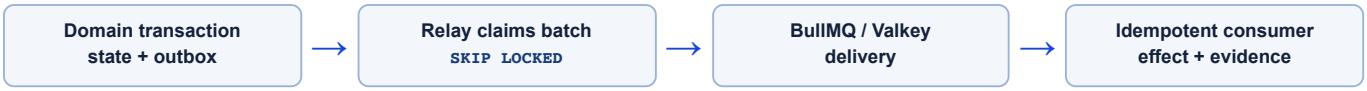


Concern	Required behavior	Common failure to reject
Data access	All operational data goes through same-origin `/api/v1`; API client maps the common error envelope.	Server component imports database package or uses an owner/service credential.
Permissions	Navigation and actions reflect permission, while API remains authoritative.	Hidden button treated as security.
Loading	Stable skeleton/progress; prevent layout jump; preserve user context.	Blank screen or spinner with no scope.
Empty	Explain why empty and the permitted next action.	Zero state that looks like a failed request.
Error	Plain-language action, safe retry and trace ID for support.	Raw stack/SQL message or infinite automatic retry.
Mutation	Stable idempotency key per user intent; disable/reconcile double submit; confirm consequential effects.	Generate a new key on every network retry.
Concurrency	Use version/precondition where stale editing matters and explain a conflict.	Last write silently overwrites approved state.
Accessibility	Keyboard flow, visible focus, semantic labels, dialogs, tables and contrast; reduced motion respected.	Clickable `div`, icon-only action without accessible name.
Evidence	Show source, observation time, confidence/verification status and human owner for AI/decision output.	Estimated benefit rendered as realised value.

Browser test matrix for each critical journey

Happy path; permission denied; wrong tenant; empty state; slow response; API 409; API 422; API 500 with trace; lost network/retry; refresh mid-flow; double click; stale version; session expiry; keyboard only; narrow viewport; long text/large number/time zone; back/forward navigation; direct URL; worker-delayed outcome; approval rejected.

17. Transactional event and worker pipeline



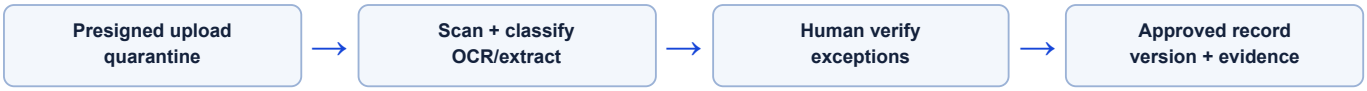
Event envelope

```
{
  "eventId": "uuid",
  "eventType": "quality.inspection.rejected.v1",
  "occurredAt": "ISO-8601",
  "tenantId": "uuid",
  "aggregate": { "type": "inspection", "id": "uuid", "version": 4 },
  "traceId": "...",
  "payload": { "contractVersion": 1, "...": "minimum data" }
}
```

Component	Contract	Metric
Producer	Writes event only with the authoritative commit; never publishes before commit.	Outbox insert failures; events per domain transaction.
Relay	Claims bounded rows, publishes with `eventId`, records attempts and retry time; lease recovers abandoned work.	Oldest pending age, batch latency, publish failures.
Queue	At-least-once delivery is expected. Retention and retry policy are explicit.	Depth, delayed count, processing age.
Consumer	Persists an idempotency/inbox record before or with its effect; duplicate is success/no-op.	Duration, duplicates, business refusals, technical failures.
DLQ	Poison event retains payload reference, error classification and replay authority.	Count and oldest age; any increase pages an owner for critical flows.
Replay	Named operator selects exact event(s), records reason and preview, then requeues under audit.	Replay success and repeated failure.

> **Correctness rule:** ledger-critical movements stay synchronous in the owning database transaction. Events notify, integrate and build projections; they do not replace the atomic inventory/accounting/approval commit.

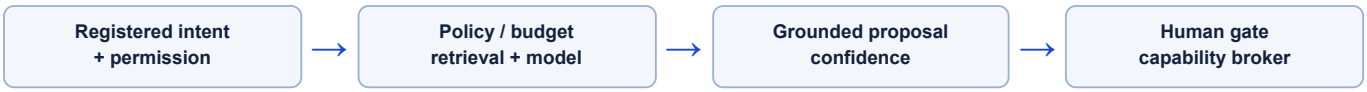
18. Document intelligence pipeline



Stage	Implementation	Security/data rule	Acceptance
Upload intent	API authorizes tenant, purpose, MIME/size and issues short-lived presigned URL plus document ID.	Object key is server-generated; client cannot choose another tenant prefix.	Wrong tenant/type/oversize is refused before use.
Quarantine	S3 receives encrypted object; event starts malware and file-structure inspection.	No download or OCR from approved area until clean result.	Malicious and malformed fixtures remain inaccessible.
Extraction	OCR/classifier/provider adapter produces fields, coordinates, confidence, model/version and cost.	Provider receives only permitted content; retention and region contract recorded.	Golden document set meets per-field thresholds.
Review	Human sees original page beside extracted fields; low-confidence/financial/legal fields require confirmation.	Reviewer permission and decision note audited.	Corrections become labeled evaluation data with privacy controls.
Commit	Validated values enter the owning business transaction; document link, checksum and version are retained.	OCR never writes directly to ledger/master tables.	Business record can be traced to exact object and reviewer.
Retention	Lifecycle, legal hold and deletion schedule by document class.	Deletion handles object versions and derived text/vector data.	Retention report and tested deletion evidence.

Initial use cases: supplier invoices/three-way match, quality certificates, purchase documents, employee receipts and controlled SOP/drawing ingestion. Start with one document type, one golden set and explicit human verification; do not begin with a generic “upload anything” platform.

19. AI, decision and agent execution pipeline



Execution stages

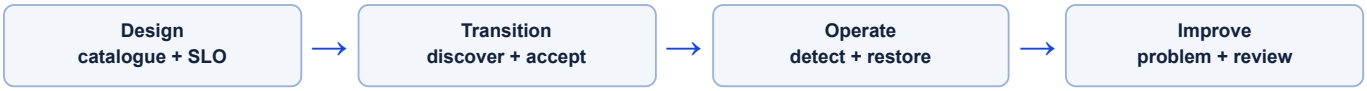
1. **Register:** capability declares owner, input/output schema, required permissions, effect class, cost ceiling and test set.
2. **Authorize:** tenant/user/mission context and opt-out/budget/kill policy are checked before retrieval or model call.
3. **Retrieve:** use fixed, tenant-authorized queries and evidence references; never let a prompt bypass data services or RLS.
4. **Generate:** provider adapter receives minimized context, hard token/time/cost limits and a structured output schema.
5. **Ground:** validate identifiers, calculations, source timestamps and domain constraints. Unsupported statements are labeled or refused.
6. **Propose:** present recommendation, consequence, evidence, confidence limits and a named human owner.
7. **Approve:** effectful capabilities pause durably. The approval grants only the exact payload and expires when context changes.
8. **Execute:** a capability broker calls the same domain API/service a human action would use; the model has no database credential.
9. **Verify:** record postcondition, action/effect IDs, actual outcome and exceptions; estimated and verified value remain separate.

Promotion ladder

Level	Behavior	Exit gate
0 · Stub	Deterministic fixtures and workflow mechanics.	Correct permissions, evidence, refusal, checkpoint and approval behavior.
1 · Shadow	Real provider runs unseen beside stub/human decision; no action.	Golden-set quality, hallucination, latency, cost and privacy thresholds.
2 · Assist	User sees proposal and sources; human performs/approves action.	Measured usefulness with safe rejection and no material control failure.
3 · Controlled action	Exact pre-approved low-risk capability executes with postcondition.	Low exception rate, reversal/recovery, continuous evaluation and kill switch.

19A. RELAY managed-service pipeline

RELAY is an operating service around the application, not a replacement for engineering. The design separates customer-facing service coordination from technical ownership so incidents, changes and reviews do not duplicate the Integration, AI Operations, Commercial or Maintenance registers.



Incident execution pipeline

1. **Ingest:** accept a user contact, XELOR event, ITSM webhook or OpenTelemetry-derived signal with tenant, source, idempotency key and observed time.
2. **Classify:** determine affected customer outcome, impact, urgency, severity, entitlement and next-update deadline. Preserve uncertain fields for human review.
3. **Route:** create one RELAY service incident and link the affected specialist record. HEXA repairs connectors/security controls; ONYX repairs AI operations; MICA owns product support; KILN owns machines; other domains keep their records.
4. **Coordinate:** maintain the response clock, escalation, timeline, decision log and authorised customer update. The specialist diagnoses and executes through its normal governed capability.
5. **Verify:** require component restoration evidence and independently evaluate the customer-facing SLI before closure.
6. **Learn:** create a problem/improvement item for repeat or material failure; include it in the next service review with owner, date and expected measure.

Target data and adapter boundaries

Boundary	Contract	Failure behavior
Telemetry → RELAY	Vendor-neutral observations: service/component, trace or metric identity, value, unit, window, source, freshness and tenant.	Quarantine invalid/unknown mapping; never invent customer impact from an unmapped component.
ITSM → RELAY	External incident/request/change ID, version, state, timestamps, participants and correlation key.	Idempotent replay; preserve conflicts for a human rather than last-write-wins.
RELAY → specialist	One attributable work item: customer impact, service clock, requested evidence and technical record link.	Escalate a missed acknowledgement; do not take over specialist credentials.
Specialist → RELAY	Diagnosis, action reference, verification evidence, residual risk and technical owner.	Service stays open when evidence is missing or the SLI remains outside objective.
RELAY → customer channel	Approved status, audience, incident/change reference, author and send idempotency key.	Queue/retry through HEXA Integration; never label a draft as sent.

MVP truth: today this pipeline is represented by typed illustrative data, a permission-gated read API, five UI screens and a verified/human-gated assurance graph. Live ITSM, telemetry, customer messaging, staffing and contractual SLOs are intentionally not claimed.

20. Knowledge graph and digital twin implementation

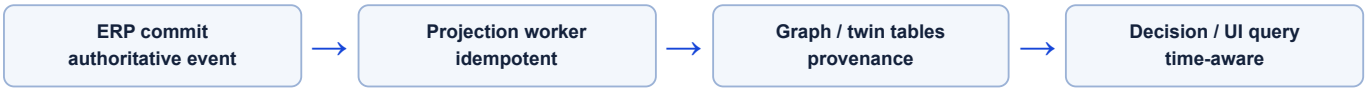
Implement both as trustworthy projections over XELOR records and events—not as a new database or a 3-D visualization project.

Knowledge graph

- Node: stable reference to an existing tenant entity or evidence object.
- Edge: typed relationship with source, observed time, valid time, confidence and creator.
- Query: permission-filtered traversal, always returning provenance.
- Start in PostgreSQL using indexed adjacency/evidence tables.
- Add a graph engine only when measured multi-hop queries cannot meet the SLO.

Digital twin

- Projection: machine/order/batch/material/quality states derived from durable domain events.
- Version: last applied event and projection schema version.
- Rebuild: deterministic replay into a new projection.
- Correction: new event or governed repair, never silent history rewrite.
- Visualization consumes the projection; it is not the twin.



Minimum edge contract

```
tenant_id, edge_id, edge_type
from_entity_type, from_entity_id
to_entity_type, to_entity_id
source_type, source_id, source_version
observed_at, valid_from, valid_to
confidence, verification_status, created_by
```

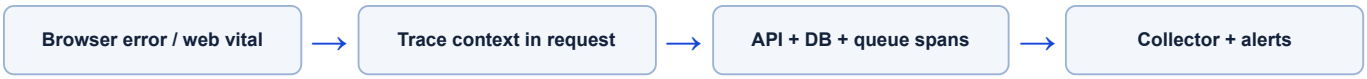
Projection operating checks

- Projection lag and last applied event are visible per tenant/partition.
- Rebuild runs into shadow tables, compares counts/checksums, then swaps after approval.
- Queries distinguish “current observed state,” “planned state” and “model-estimated state.”
- A missing projection never changes ledger truth; user receives a stale/unavailable indication.

21. Health, observability and service objectives

Probe	What it means	Dependencies	Platform action
`/health/live`	Process event loop/server is alive.	None; no DB query.	Restart only after repeated failure.
`/health/ready`	Task can serve this runtime role safely.	DB connection, compatible migration range; required broker for broker-dependent routes.	Remove from target group; do not restart on a brief downstream outage unless policy requires.
Startup	Boot/config/instrumentation completed.	Validated config and initialized resources.	Allow slow boot without premature liveness restart.

Telemetry path



Use JSON logs with timestamp, level, service, environment, release, trace/span IDs, route template, safe tenant pseudonym, outcome code and duration. Redact authorization headers, cookies, passwords, secrets, full prompts/documents, personal financial data and arbitrary request bodies.

Initial SLOs—confirm with pilot users

User journey	SLI	Initial objective	Alert signal
Authenticated read API	Successful eligible requests and p95 latency.	99.9% monthly; p95 < 500 ms for normal list/detail requests.	Fast/slow burn of error budget, not one isolated 500.
Ledger mutation	Definitive accepted/refused response without ambiguity.	99.95% processing availability; p95 < 1.5 s excluding deliberate external waits.	Ambiguous result, transaction error or elevated conflict anomaly.
Outbox delivery	Age of oldest undispatched critical event.	99% under 60 s; no critical event stranded 5 min.	Age and DLQ, not only worker CPU.
Approval visibility	Committed pending approval visible to authorized inbox.	99% within 30 s.	Projection/queue lag or mismatch count.
Restore	Successful rehearsal within approved RPO/RTO.	Initial RPO ≤15 min, RTO ≤4 h; tighten from business impact.	Missed drill or failed evidence.

22. Verification strategy and release test matrix

Layer	Runs	Must prove
Static	Every PR, seconds/minutes.	Type, lint, import boundary, manifest and config schema correctness.
Unit/property	Every PR.	Business invariants, monetary/quantity boundaries, state machines, refusal codes and deterministic AI tools.
Database	Every DB PR against PostgreSQL 17.	Fresh/upgrade migration, RLS/force, constraints, concurrent write conflicts, idempotency and query plan threshold.
API contract	Every PR.	Auth, permission, validation, error envelope, pagination, idempotency and version compatibility.
Integration	Every PR for affected paths.	Real PostgreSQL/Valkey/Keycloak, outbox/worker and cross-module transactions.
Browser	Critical per PR; full before release/nightly.	20 state/interaction variants listed in §16, using normal OIDC as well as demo separately.
Acceptance	Staging release.	Current 99-item matrix plus investor Northstar, approval and recovery stories.
Performance	Baseline before pilot; scheduled/regression threshold.	API latency, DB pool, hot tenant/list queries, lock behavior, worker throughput and resource saturation.
Security	Continuous scans; threat/pentest before customer data.	Tenant escape, IDOR, auth/session, SSRF/upload, injection, secret, dependency and privilege escalation controls.
Resilience	Pre-pilot and quarterly.	Task loss, DB failover, broker interruption, poison message, provider timeout and partial connector response.
Recovery	Scheduled restore drill.	Backup can restore into a new environment, migrate, reconcile and meet RPO/RTO.

Release evidence bundle

Commit and PR; reviewers; dependency lock hash; test results; migration checksums; SBOM/scan/signature; Web/runtime digests; IaC plan; staging deployment and acceptance results; production approver; deploy timeline; smoke/SLO outcome; rollback/incident reference if any.

> **Verification principle:** count scenarios only when their assertion is meaningful. Fifty variations that all check “page loads” are weaker than ten tests that prove authority, atomicity, replay, refusal and recovery.

23. Rollback, backup and disaster recovery

Deployment rollback decision tree

Observation	Immediate action	Data action
New tasks fail readiness before traffic.	ECS circuit breaker stops/rolls to last completed task definition.	Inspect migration compatibility; no schema reverse by default.
Error/latency alarm during rolling deployment.	Automatic rollback when alarm policy is linked; freeze further promotion.	If additive schema, leave it. Forward-fix if data was written in new form.
Application bug with compatible schema.	Redeploy last known-good image digest.	Reconcile any affected transactions using audit/idempotency evidence.
Bad migration before new writes.	Stop rollout; execute a reviewed forward correction.	Restore only if forward correction is unsafe and loss window is accepted.
Corruption or accidental destructive write.	Contain writes, preserve evidence, declare incident.	Point-in-time restore creates a new RDS instance/cluster; compare and reconcile before cutover.
Regional outage.	Invoke DR authority and DNS/runbook after data recovery condition is known.	Restore/replicate into recovery region and verify tenant/audit integrity before traffic.

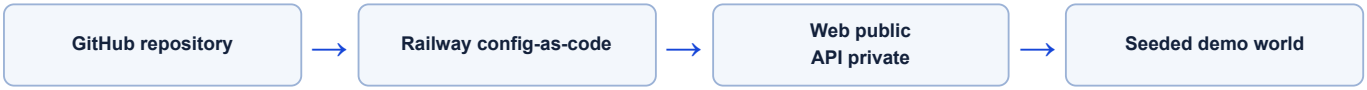
Restore drill

1. Select an approved recovery point and restore RDS into a new isolated environment—never over the source.
2. Restore/attach required S3 object versions and configuration; rotate credentials for the drill environment.
3. Deploy the matching historical image digests, then apply only compatible migrations.
4. Run schema, RLS, audit-chain, tenant counts, ledger balances, outbox and document-reference reconciliation.
5. Execute representative read/mutation/approval flows with synthetic accounts.
6. Measure actual RPO/RTO; record gaps, owner and due date; destroy the isolated environment through reviewed cleanup after evidence retention.

> **Backup is not recovery:** PITR capability is only proven when a team can restore, validate and make a safe cutover decision inside the business target.

24. Railway public-demo pipeline

Keep Railway as a disposable investor channel. It is intentionally simpler than production and must stay unable to receive real data.



Service	Config	Start dependency	Public?
`xelor-postgres`	`postgres.railway.json`; volume.	Role/database initialization.	No.
`xelor-valkey`	`valkey.railway.json`.	Valkey starts.	No.
`xelor-api`	`api.railway.json`.	Wait DB → migrate → API live → versioned seed → 97 checks → ready.	No.
`xelor-worker`	`worker.railway.json`.	Wait for API ready and Valkey.	No.
`xelor-web`	`web.railway.json`.	Wait for API ready; same-origin rewrite.	Yes.

Deploy order

1. Create shared strong, different owner/app database secrets.
2. Add the same GitHub repo as exactly five named Railway services and attach the supplied config paths.
3. Attach the PostgreSQL volume at `/var/lib/postgresql/data`.
4. Deploy PostgreSQL and Valkey, then API, then Worker and Web.
5. Create a domain only for Web; check same-origin `/api/v1/health` and the investor journey.
6. Optionally add `xelor.kisancred.com` as Web custom domain after the generated Railway URL works.

> **Isolation policy:** public-demo flags are acceptable only because the dataset is disposable and integrations are disabled. Do not connect this environment to customer identity, bank, GST, email, production ERP, documents or analytics destinations.

Demo release checks

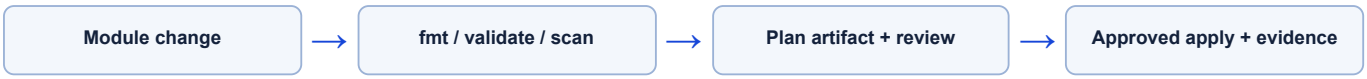
Local five-container parity; cold database boot; second boot without duplicate seed; public URL and API proxy; all presenter personas; 99-item matrix; worker event delivery; mobile/desktop display; suspend/resume; no sign-in page; no non-Web public service.

25. AWS build sequence—exact order

Wave	Implementation	Exit criterion
0 · Decisions	Confirm AWS Organization/accounts, Mumbai primary, approved recovery region, DNS authority, pilot RPO/RTO, data classes and named owners.	Architecture decision records and risk owners approved.
1 · Bootstrap	Create OpenTofu state bucket, versioning, KMS, state locking, GitHub OIDC providers/roles and break-glass roles through a small audited bootstrap.	A clean runner can plan with short-lived identity; state is encrypted/locked.
2 · Network	VPC across at least two AZs; public edge subnets, private application/data subnets, NAT/VPC endpoints, route tables, flow logs and security groups.	Only edge is reachable publicly; connectivity tests match the matrix.
3 · Data	RDS Multi-AZ, parameter/option/subnet groups, backups/PITR, alarms; managed Valkey; S3/KMS buckets and lifecycle.	Encrypted private services; restore and failover smoke succeeds.
4 · Platform	ECR, ECS cluster, task roles, CloudWatch logs, OTel Collector, ALB target groups/listeners, autoscaling and deployment alarms.	Hello/health workloads deploy by digest with circuit breaker.
5 · Identity	Keycloak service/database, realm-as-code/export, MFA/session/brute-force policy, OIDC client and application mapping.	Normal user login and token/tenant/permission negative tests pass.
6 · Application	Migration task; API/Worker/Web services; same-origin routing; config assertions; readiness and graceful shutdown.	Staging completes 97 checks and critical browser journeys.
7 · Operations	SLO dashboards, alert routing, runbooks, on-call, vulnerability flow, backup/restore, incident exercise and cost budgets.	Named operators complete a game day without repository author assistance.
8 · Production	Production account plan/apply, approved release promotion, DNS/TLS, pilot tenant onboarding and heightened observation.	Go-live checklist signed; rollback/restore authority on call.

Region note: Mumbai (`ap-south-1`) is the recommended primary for India-facing latency and residency discussions. Hyderabad (`ap-south-2`) is a candidate recovery region; final selection must verify service availability, contractual residency and business recovery cost at implementation time.

26. OpenTofu and infrastructure change flow



State and account rules

- Separate AWS accounts and OpenTofu state for staging and production. Do not use workspaces alone as the isolation boundary.
- Use an encrypted, versioned S3 backend with state locking and short-lived OIDC credentials. Restrict read access because state can contain sensitive values.
- Pin provider/module versions and commit lock files. Modules expose small typed inputs and meaningful outputs; environment roots compose them.
- Pull requests run formatting, validation, static security checks and a read-only plan. Apply runs only from protected environments.
- Detect drift on a schedule. Import deliberate console changes or revert them; never let drift become the undocumented source of truth.

Module boundaries

Module	Owns	Does not own
`network`	VPC, subnets, routes, endpoints, flow logs, baseline security groups.	Application task policies.
`edge`	Certificates, CloudFront, WAF, ALB/listeners/target groups.	DNS delegation outside the managed zone.
`ecs-service`	Task/service, autoscaling, health and deployment alarms.	Image build or schema migration.
`rds`	Database, subnet/parameter groups, monitoring, backup/restore policy.	Application schema.
`telemetry`	Logs, collector, dashboards, alarms and destinations.	Business metric definitions in code.

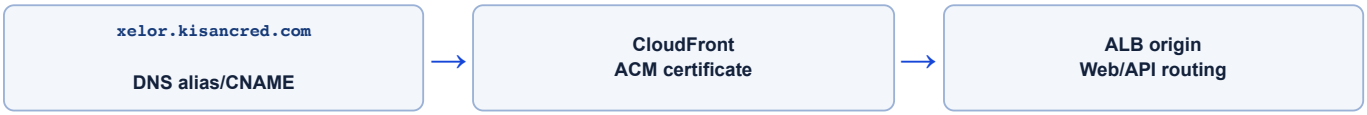
Destroy protection: production databases, KMS keys, evidence buckets and state backends need deletion protection and explicit two-person break-glass procedure. A routine pipeline must not contain a broad automatic destroy path.

27. Domain, TLS and `xelor.kisancred.com`

Public demo on Railway

1. First prove the Railway-generated `*.up.railway.app` URL.
2. Add `xelor.kisancred.com` only to `xelor-web`.
3. Railway provides a DNS target; create the exact CNAME at the authoritative DNS provider.
4. Do not change the apex `kisancred.com` records. Keep proxying off until Railway validates and issues TLS if the DNS provider has a proxy mode.
5. Verify certificate, redirect to HTTPS, Web route, same-origin API route and no public ports on API/data services.

AWS production



- Request ACM certificate in the region required by CloudFront and validate through controlled DNS.
- Set HTTP→HTTPS redirect, HSTS only after the HTTPS path is stable, secure cookies and the response headers already begun in Next config.
- Use low DNS TTL during initial cutover. Preserve the previous endpoint until the observation window ends.
- Prepare rollback: switch DNS/alias to the known-good endpoint only when data/schema compatibility is understood; DNS alone cannot undo database writes.

Cutover checks

✓	Certificate chain, hostname and expiry monitoring pass.
✓	`/`, deep link, static asset and `/api/v1/health/ready` work over HTTPS.
✓	OIDC redirect/logout URLs exactly match the new origin.
✓	No CORS wildcard, mixed content, internal hostname or server banner is exposed.
✓	WAF logs, request traces and error alerts show the synthetic cutover requests.

28. Failure-mode review

Failure	User-visible behavior	Detection	Recovery
API task dies	Other AZ task serves; in-flight mutation safely retryable with same key.	Target health, task exit, 5xx/latency.	ECS replaces task; investigate trace/crash.
PostgreSQL failover	Brief errors/latency; no false success.	RDS event, connection/error rate.	Pool reconnect with bounded jitter; reconcile ambiguous mutations by idempotency key.
Valkey unavailable	Synchronous ERP continues where designed; async effects delayed and shown as pending.	Queue connection, oldest outbox age.	Reconnect, relay durable outbox, inspect duplicates/DLQ.
Worker poison event	One integration/effect delayed; unrelated work continues.	Retry count/DLQ and event age.	Classify, fix, audited replay exact event.
Keycloak outage	New sign-ins/refresh fail; existing valid token behavior follows policy.	OIDC endpoint and auth error rate.	Restore IdP service/database; no insecure auth bypass.
AI provider timeout	Deterministic fallback/refusal; no partial action.	Provider latency/error/budget.	Circuit/open policy, alternate approved provider or human workflow.
Document scanner down	Upload remains quarantined/pending.	Oldest quarantine age.	Resume scan; never release unscanned object.
Bad deployment	Old tasks remain or return automatically.	ECS circuit breaker/CloudWatch alarms.	Last completed task definition; forward-fix data if needed.
Tenant-isolation regression	Request refused or release stopped—never accepted as degraded behavior.	RLS/IDOR tests, anomaly/security alert.	Contain, revoke sessions if required, incident/privacy assessment and audited correction.

Design target: every dependency failure produces one of three explicit outcomes—committed, refused or pending/reconcilable. “Probably succeeded” is not acceptable for money, stock, quality disposition or approval.

29. Twelve-week implementation plan

Weeks	Workstream	Concrete output	Gate
1–2	Release foundation	CODEOWNERS, protected checks, `ci.yml`, container hardening, SBOM/scan, release manifest.	PR cannot merge with a broken current test suite.
1–2	Runtime corrections	Same-origin runtime API route, environment schema/startup assertions, live/ready/startup probes, SIGTERM drain.	Same Web digest in two local configs; task leaves load balancer gracefully.
3–4	Cloud bootstrap	AWS accounts/roles, OIDC, state backend, core OpenTofu modules, staging network/ECR/ECS skeleton.	Reviewed plan/apply with no static AWS key.
5–6	Managed state	RDS Multi-AZ, Valkey, S3/KMS, Secrets Manager, migration task and connection budget.	Migration and restore/failover rehearsal.
5–7	Identity	HA Keycloak/approved IdP, MFA/session policy, realm config, tenant mapping tests.	OIDC negative matrix and admin recovery procedure.
7–8	Staging delivery	Deploy-staging workflow, ECS circuit breaker/alarms, critical acceptance and browser pipeline.	One-click repeatable staging release from `main`.
8–9	Observability	OTel, structured/redacted logs, SLO dashboards, paging and trace-linked support view.	Injected API/worker failure produces actionable alert and trace.
9–10	Production promotion	Protected production workflow, same-digest verification, rollback/incident runbooks.	Game day rolls back an intentionally bad release.
10–11	Recovery/security	PITR restore drill, threat model, tenant/IDOR test, dependency/secret remediation flow.	RPO/RTO evidence and no unresolved critical finding.
12	Pilot readiness	Capacity baseline, support ownership, DNS/TLS cutover plan and pilot go/no-go dossier.	Joint engineering/product/security/operations sign-off.

Calendar assumes a small focused team and one pilot region. Identity procurement, enterprise integration certification or compliance assessment can extend it; do not hide those dependencies inside an engineering estimate.

30. Team ownership and operating model

Role	Owns	Must review	Primary evidence
Senior/backend lead	Domain boundaries, transaction design, API contracts, DB change policy.	Ledger writes, migrations, auth/effect code.	Invariant tests, migration/compatibility proof.
Frontend/full-stack	Web shell, API integration, interaction/accessibility, browser tests.	Response contract and permission-facing changes.	State matrix, Playwright and accessibility results.
Platform/SRE	IaC, pipelines, compute/network, telemetry, recovery and on-call tooling.	Runtime config, health, scaling and dependency changes.	Plan/apply, deploy manifest, SLO and drill report.
Security/identity	Threat model, OIDC policy, secrets, tenant test, incident/privacy controls.	New data path, external connector, AI/provider and privilege.	Risk decision, scan/pentest and access review.
QA/product domain	Business acceptance, golden scenarios, refusal/recovery expectations.	User-facing workflow and value claim.	Signed scenario result and known-limit register.
Managed-service owner / manager	Catalogue, transition, coverage, incident/change coordination, service reviews and improvement.	Customer outcome/SLO changes, support boundaries, escalation and external communication.	Acceptance record, incident/change timeline, service report and improvement register.
Service desk / operations lead	Contact intake, event triage, response clocks, specialist hand-offs and customer update cadence.	Runbooks, paging, major incident and post-restoration closure.	Shift/on-call evidence, incident log, update history and restoration proof.
Release approver	Production go/no-go against evidence.	Migration, risk, rollback, incident coverage.	GitHub Environment approval and release record.

Minimum staffing for this plan

One experienced backend/architecture owner, one full-stack/frontend owner, and one platform engineer can establish the foundation, with part-time product/QA and security/identity review. A single person may cover two roles in an MVP team, but approvals and production infrastructure should still receive independent review.

Operating cadence

- Daily: failed release checks, critical alerts, queue age and security notifications.
- Weekly: dependency updates, error-budget and slow-query review, demo reset/rebuild proof.
- Monthly: access/secret/cost review, SLO report, recovery action tracking.
- Quarterly: restore/game day, permission/SoD review, threat model and external provider review.
- Per release: compatibility, migration, same-digest, acceptance and observation evidence.

31. “Ready to build” acceptance checklist

	Architecture and delivery condition
✓	Target environment/account/region and DNS authority are named.
✓	Protected `main` requires the complete CI success gate.
✓	GitHub uses OIDC; staging and production roles are separate and subject-restricted.
✓	Web/runtime images build once, are scanned/signed and promote by digest.
✓	Web makes same-origin API calls without an environment-specific public build variable.
✓	Live/ready/startup semantics and graceful task draining are exercised.
✓	Production migrations run once with owner authority and use expand/contract compatibility.
✓	API/Worker use only the restricted RLS-enforced application database role.
✓	Production cannot start with demo identity flags or default credentials.
✓	RDS, Valkey, S3 and application tasks are private and security-group scoped.
✓	Every critical request can be traced across API, database and worker without leaking sensitive data.
✓	Deployment alarms can stop/rollback; last known-good digests are retained.
✓	PITR restore and application reconciliation have been proven in an isolated environment.
✓	Critical happy, refusal, duplicate, concurrency, failure and recovery paths pass.
✓	Named humans own release, incident, data recovery, identity and customer communication decisions.

Go/no-go rule: production readiness is a body of evidence, not a confidence statement. If a mandatory item has no named owner, test result or runbook, it is not complete.

32. Developer review of this playbook

Backend developer review

Question	Finding	Resolution in this playbook
Can I tell where business logic and transaction ownership belong?	Yes.	Platform invariant → owning application service → tenant transaction → audit/outbox, §§14–15.
Can I add a table without weakening tenancy?	Yes, with explicit checks.	Migration pipeline and tenant-table proof, §11.
Can I deploy a DB change safely with mixed task versions?	Yes, if expand/contract is followed.	Compatibility releases and service order, §§10–11.
What happens to duplicate requests/events?	Defined.	Stable mutation key and idempotent consumer/inbox, §§15 and 17.
Who can migrate the production database?	Only the one-shot task role.	Owner secret never reaches API/Worker, §§11–13.
How do I debug a failure?	Traceable contract is defined.	Error `traceId`, structured/redacted telemetry and SLOs, §§15 and 21.

Full-stack developer review

Question	Finding	Resolution in this playbook
Do I know how Web reaches API in every environment?	Yes; current build-time coupling is explicitly corrected.	Same-origin ALB routing/runtime local rewrite, §§3 and 5.
Do I know the required UI states and negative paths?	Yes.	Screen state and 20-scenario browser matrix, §16.
Can I ship the same Web artifact through staging/production?	Yes after the named prerequisite.	Remove public environment API origin, §§2, 3 and 7.
Can I understand CI/CD without undocumented steps?	Yes.	Named files, jobs, sequence, roles and proof bundle, §§6–10.
Can I tell whether a screen is “done”?	Yes.	Cross-layer definition of done and acceptance checklist, §§14, 16 and 31.

Remaining decisions requiring the actual delivery team: confirmed pilot RPO/RTO and SLOs, final recovery region, managed IdP versus operated Keycloak, compliance scope, expected concurrency/data volume, notification/integration providers and monthly cloud budget. This document makes their decision points explicit rather than inventing answers.

33. Cross-functional review

Perspective	Question asked	Finding	Implemented answer
Architecture	Does this add distributed complexity before evidence?	No.	Modular monolith remains; extraction triggers are measurable, §§1 and 4.
Database	Can schema and application versions overlap safely?	Yes with discipline.	One-shot migration, advisory lock, expand/migrate/contract and forward-fix policy, §11.
Security	Can an environment or pipeline silently gain broad authority?	Controls are explicit.	OIDC subject restrictions, separate roles/accounts, secret separation and startup refusal, §§9 and 12.
Tenant privacy	Is tenant filtering only an application convention?	No.	Verified identity, tenant transaction helper and FORCE RLS with release checks, §§5 and 11.
SRE	Can a bad release fail before affecting all traffic?	Yes.	Readiness, rolling overlap, circuit breaker, alarms and known-good digest, §§10, 21 and 23.
Recovery	Is backup restoration actually testable?	Yes.	Scheduled isolated PITR restore and reconciliation sequence, §23.
QA	Are correctness and recovery tested beyond happy paths?	Yes.	Layered matrix including permission, duplicate, concurrency, failure, replay and restore, §22.
Accessibility	Can a keyboard/small-screen user complete critical work?	Required but must remain a release gate.	Explicit UI states and browser matrix, §16.
AI governance	Can a model directly write to ERP or invent authority?	No.	Registered capability, fixed retrieval, human gate and domain broker, §19.
Cost	Does the MVP require EKS/Kafka/graph DB/warehouse?	No.	Managed essentials first; adoption triggers defer optional platforms, §4.
Support	Can a normal developer trace and explain one failure?	Yes after telemetry work.	Stable error code/trace ID, structured redaction and evidence bundle, §§15 and 21.
Investor demo	Can public access stay simple without weakening production?	Yes only through isolation.	Railway public demo is disposable; production startup rejects its flags, §§3, 12 and 24.

Review conclusion: the design is implementable by a small full-stack team because it strengthens the existing architecture instead of replacing it. The largest remaining risks are operational execution, identity ownership, restore proof and pilot-specific business requirements—not an absent technology category.

34. Exact developer command runbook

First local setup

```
corepack enable
pnpm install --frozen-lockfile
test -e .env || cp .env.example .env # review local values; never use them in cloud
pnpm infra:up
pnpm db:migrate
pnpm demo:seed
pnpm demo:northstar
```

Run the application in separate terminals

```
# Terminal 1 — supporting services were started by pnpm infra:up

# Terminal 2 — API on http://localhost:3000
pnpm dev

# Terminal 3 — Web on http://localhost:3001
pnpm --filter @ind-core/web dev

# Terminal 4 — Worker (build its runtime first)
pnpm --filter @ind-core/api build
pnpm --filter @ind-core/api worker
```

Run the current repository gates

```
pnpm lint
pnpm typecheck
pnpm test
pnpm --filter @ind-core/web module-check
pnpm db:rls-check
pnpm db:perm-check
pnpm demo:verify
pnpm --filter @ind-core/web exec playwright test --list
```

Rebuild the disposable demo world

```
pnpm demo:rebuild
# This intentionally deletes/recreates demo data. Never aim it at production.
```

Local Railway parity

```
export POSTGRES_PASSWORD="$(openssl rand -hex 24)"
export APP_DATABASE_PASSWORD="$(openssl rand -hex 24)"
docker compose -f infra/railway/docker-compose.demo.yml build
docker compose -f infra/railway/docker-compose.demo.yml up
```

The future AWS deployment commands belong inside reviewed GitHub workflows and OpenTofu roots, not in a developer copy/paste procedure. Their task definitions, cluster/subnet/security-group identifiers and release digests must come from environment outputs and the signed release manifest.

35. Source and verification record

The implementation recommendations were checked against the live repository and current primary platform documentation. Provider behavior and pricing should be rechecked at implementation time.

Repository sources

- `package.json` and four workspace package manifests.
- `apps/web/next.config.ts`, Web API/session implementation and Playwright suite.
- `apps/api` controllers, identity/tenant guards, health code, Worker and demo verification scripts.
- `packages/platform` module manifests, services, Agent OS and boundary rules.
- `packages/platform/src/managed-services`, RELAY API/UI module and managed-service assurance graph.
- `packages/db` schema, 64 SQL migrations, tenant helper, migration/RLS/permission scripts.
- `infra/docker-compose.yml`, `infra/railway` Dockerfiles/config/start scripts and deployment guide.

Primary official references

- GitHub Docs — Configuring OpenID Connect in Amazon Web Services
- GitHub Docs — OpenID Connect reference
- AWS — Amazon ECS deployment circuit breaker
- AWS — ECS deployment failure detection and rollback
- AWS — ECS deployment strategies
- AWS — Point-in-time recovery for Multi-AZ DB clusters
- AWS — Secrets Manager rotation
- OpenTelemetry — JavaScript instrumentation status
- ISO/IEC 20000-1 — service management systems
- NIST SP 800-61 Rev. 3 — incident response
- Google SRE — Service Level Objectives
- OpenTofu — S3 backend, locking and encryption
- Railway — Config as code reference

Document validation to perform after rendering

✓	HTML headings, tables, diagrams, links and page order programmatically inspected.
✓	PDF rendered with print backgrounds and A4 sizing; file/page metadata checked.
✓	Representative architecture, pipeline, implementation and review pages visually inspected.
✓	Repository-specific claims cross-checked; future recommendations labeled as target decisions.
✓	Backend, full-stack, architecture, database, security, SRE, recovery, QA, accessibility, AI, cost and support reviews completed; unresolved business choices stated explicitly.

This is an engineering implementation blueprint, not a certification, penetration-test report, legal opinion or guarantee of uninterrupted service.